

Flatcar First Boot & Provisioning

Flatcar is configured at **first boot only** via declarative config. The primary toolchain is: **Butane YAML** → **Ignition JSON** → **applied by Ignition in initramfs**.

Source section: `fb-provision/` (renamed from `provisioning/` in the refactored branch).

Toolchain Overview

```
Butane YAML (human-writable)
  ↓ butane / docker run quay.io/coreos/butane:latest
Ignition JSON (machine-readable)
  ↓ passed as userdata / cloud metadata / kernel param
Ignition (runs in initramfs, first boot only)
  ↓
Configured Flatcar node
```

Ignition (`fb-provision/ignition/`)

Ignition runs in the **initramfs** on first boot only — before systemd starts. Capabilities: partition disks, format filesystems, write files, configure users, install systemd units.

Trigger mechanism:

- GRUB checks EFI System Partition for `flatcar/first_boot`
- Sets `flatcar.first_boot=detected` kernel parameter
- Ignition runs, then deletes the flag file
- Subsequent boots skip Ignition

Config sources (one at a time):

- Cloud provider instance userdata
- Network URL (`ignition.config.url` kernel parameter)
- QEMU fw_cfg (`opt/org.flatcar-linux/config`)
- VMware guestinfo (`ignition.config.data`)

PXE/iPXE: Must explicitly set `flatcar.first_boot=1` as a kernel parameter (GRUB flag check doesn't apply).

Re-provisioning: Use `flatcar-reset` (Alpha 3535.0.0+) to selectively clean rootfs and re-trigger Ignition:

```
sudo flatcar-reset --keep-machine-id --keep-paths '/etc/ssh/ssh_host_.*'
/var/log
sudo systemctl reboot
```

Key differences from cloud-init:

- JSON format — no YAML type-coercion bugs
- Versioned config schema
- Runs exactly once — no variable substitution, no drift

Butane Config Transpiler (fb-provision/butane/)

Renamed from `config-transpiler/` in the refactored branch. Butane is the human-writable YAML layer that transpiles to Ignition JSON.

Transpiling:

```
cat config.yaml | docker run --rm -i quay.io/coreos/butane:latest >
ignition.json
# or with the butane binary:
butane < config.yaml > ignition.json
```

Why two steps? Catches config errors before boot, and keeps the boot path simple (JSON-only, no YAML parsing in initramfs).

Common Butane sections:

Section	Purpose
<code>systemd.units</code>	Define/enable/disable systemd units
<code>storage.files</code>	Write arbitrary files
<code>storage.links</code>	Create symlinks (e.g., to disable sysexts)
<code>passwd.users</code>	Create users, add SSH keys
<code>kernel_arguments</code>	Set kernel cmdline args

Example — NGINX container as a systemd service:

```
variant: flatcar
version: 1.0.0
systemd:
  units:
```

```

- name: nginx.service
  enabled: true
  contents: |
    [Unit]
    Description=NGINX example
    After=docker.service
    Requires=docker.service
    [Service]
    ExecStart=/usr/bin/docker run --name nginx1 --net host
docker.io/nginx:1
    Restart=always
    [Install]
    WantedBy=multi-user.target

```

Example — add SSH key to core user:

```

variant: flatcar
version: 1.0.0
passwd:
  users:
    - name: core
      ssh_authorized_keys:
        - ssh-rsa AAAAB.....xyz user@host

```

Container Linux Config (fb-provision/cl-config/)

The older CLC format (YAML, transpiled via the CL Config Transpiler). Still supported, but Butane is the current standard. CLC is required for LTS-2022 (Butane not supported there).

Migrating from cloud-config to CLC: [fb-provision/cl-config/from-cloud-config](#).

Infrastructure / Terraform (fb-provision/infrastructure/)

Terraform-based provisioning. Butane/Ignition configs are passed via Terraform's `user_data` field to cloud provider resources.

Image Customization (fb-provision/customize-image/)

Covers baking custom content into a Flatcar image at build time — pre-populating `/etc` or bundling sysext images before deployment. See also `sysext-bakery`'s `bake_flatcar_image.sh`.

System Extensions / sysext (sys-ext/)

Now a top-level section (was under `provisioning/sysextd/` in main branch). Sysext images extend the immutable base OS by mounting a `/usr` overlay — the recommended way to add custom binaries without modifying the base image.

Official sysext extensions (configure via `/etc/flatcar/enabled-sysext.conf`):

Name	Available since	Default
containerd-flatcar	3794.0.0	Yes (opt-out)
docker-flatcar	3794.0.0	Yes (opt-out)
podman	3941.0.0	No
zfs	3913.0.0	No
python	4012.0.0	No
incus	4285.0.0	No
nvidia-drivers-*	4344.0.0	No
overlaybd	4426.0.0	No
oem-*	3760.2.0	Yes (opt-out)

Disabling Docker/containerd (e.g., to use custom versions for Kubernetes):

```
variant: flatcar
version: 1.0.0
storage:
  links:
    - path: /etc/extensions/docker-flatcar.raw
      target: /dev/null
      overwrite: true
    - path: /etc/extensions/containerd-flatcar.raw
      target: /dev/null
      overwrite: true
```

Sysext image format:

- Named `NAME.raw` (squashfs recommended) or plain `NAME/` directory
- Placed in `/etc/extensions/` or `/var/lib/extensions/`
- Metadata file: `usr/lib/extension-release.d/extension-release.NAME` containing `ID=flatcar` and either `SYSEXT_LEVEL=1.0` (static binaries) or `VERSION_ID=X.Y.Z` (dynamically linked)
- Binaries in `usr/bin/`, systemd units in `usr/lib/systemd/system/`

Runtime reload:

```
systemctl restart systemd-sysex
```

Community sysexts: <https://github.com/flatcar/sysex-bakery>

Updating custom sysexts: Use `systemd-sysupdate` (available since 3510.2.0).